

VETTORIALIZZAZIONE CON NUMBA

◆ Idea

👉 In numpy esistono funzioni (**ufuncs**) che:

- operano su interi array
- anche se definite su singoli elementi

👉 iterano automaticamente sugli elementi

✳ applicano la funzione a tutto il vettore

👉 possiamo ottenere lo stesso comportamento con Numba

✳ usando **@vectorize**

◆ @vectorize

👉 Permette di:

- definire una funzione su scalari
- applicarla automaticamente a vettori



Codice:

```

from numba import vectorize, guvectorize, jit, float64
import numpy as np
import timeit

def standard_fma(va, vb, vc):
    out = np.empty_like(va)
    for i in range(va.shape[0]):
        out[i] = va[i] * vb[i] + vc[i]
    return out

@jit
def jit_fma(va, vb, vc):
    out = np.empty_like(va)
    for i in range(va.shape[0]):
        out[i] = va[i] * vb[i] + vc[i]
    return out

@vectorize([float64(float64, float64, float64)], target='cpu')
def vectorized_fma(a, b, c):
    return a * b + c

@guvectorize([(float64[:,], float64[:,], float64[:,], float64[:,])],
              '(n),(n),(n)->(n)')
def guvectorized_fma(a, b, c, out):
    for i in range(a.shape[0]):
        out[i] = a[i] * b[i] + c[i]

@guvectorize([(float64[:,], float64[:,], float64, float64[:,])],
              '(n),(n),()->(n)')
def guvectorized_fma_fixed_c(a, b, c, out):
    for i in range(a.shape[0]):
        out[i] = a[i] * b[i] + c

```

◆ Spiegazione del codice

👉 standard_fma :

- ciclo esplicito Python

- versione base

👉 `jit_fma` :

- stessa logica
- compilata con **@jit**

👉 `vectorized_fma` :

- funzione scalare
- trasformata in funzione vettoriale

✳️ niente ciclo esplicito

👉 `guvectorized_fma` :

- lavora su vettori
- output passato come parametro

👉 `guvectorized_fma_fixed_c` :

- `c` è scalare
- output vettoriale

◆ Signature

👉 `[float64(float64, float64, float64)]` :

- tipi input/output

👉 possiamo:

- aggiungere più signature
- per diversi tipi

👉 comportamento:

- con signature → compilazione immediata (*eager*)

- senza signature → compilazione alla chiamata (*lazy*)

◆ Tipo di esecuzione (target)

👉 con **@vectorize** possiamo specificare:

- target='cpu' → esecuzione standard
- target='parallel' → multi-thread su CPU
- target='cuda' → GPU (CUDA)

✳️ stessa funzione, modalità diverse

◆ Limiti di @vectorize

👉 funziona bene quando:

- input stessa dimensione
- output stessa dimensione

👉 problema:

- alcuni parametri possono essere scalari
- output può avere forma diversa

✳️ serve qualcosa di più generale

◆ @guvectorize

👉 Versione più generale di @vectorize

👉 caratteristiche:

- lavora su vettori e scalari
- NON restituisce valore
- output passato come parametro

👉 bisogna specificare:

- tipi
- forma degli argomenti

◆ Forma degli argomenti

👉 '(n), (n), (n) -> (n)' :

- tutti vettori

👉 '(n), (n), () -> (n)' :

- () → scalare

◆ Differenze principali

👉 @vectorize :

- elemento per elemento
- più semplice

👉 @guvectorize :

- lavora su blocchi (vettori)
- più flessibile

◆ Esecuzione e confronto



Codice:

```
n = 1_000_000
rng = np.random.default_rng()
a = rng.random(n)
b = rng.random(n)
c = rng.random(n)
out = np.empty_like(a)

print(f"Esecuzione di standard_fma: {timeit.timeit(lambda: standard_fma(a, b, c), number=1)}")
print(f"Esecuzione di jit_fma: {timeit.timeit(lambda: jit_fma(a, b, c), number=1)}")
print(f"Esecuzione di vectorized_fma: {timeit.timeit(
    lambda: vectorized_fma(a, b, c), number=1)}")
guvectorized_fma(a, b, c, out)
print(f"Esecuzione di guvectorized_fma: {timeit.timeit(
    lambda: guvectorized_fma(a, b, c, out), number=1)}")

print(f"Esecuzione di guvectorized_fma with fixed c: {timeit.timeit(
    lambda: guvectorized_fma_fixed_c(a, b, 0.5, out), number=1)}")
```

◆ Osservazione

👉 il codice confronta:

- Python puro
- compilato
- vettorializzato

✳️ mostra differenze di prestazioni

Ora vediamo come parallelizzare i cicli con **Numba**

PARALLELIZZARE I CICLI CON NUMBA

◆ Idea

👉 Con il decoratore **@jit** possiamo compilare il codice

👉 Possiamo anche dire a Numba di:

✳ parallelizzare i cicli

◆ @jit(parallel=True)

👉 Possiamo scrivere:



Codice:

```
@jit(parallel=True)
```

👉 indica a Numba di:

- parallelizzare il codice (stile OpenMP)

👉 Problema:

✳ Come fa a sapere cosa parallelizzare?

◆ prange

👉 Numba fornisce:

✳ **prange**

👉 prange :

- sostituisce `range`
- indica che il ciclo può essere parallelizzato

👉 Se NON usiamo `parallel=True` :

✳ `prange` si comporta come un normale `range`

◆ Esempio (dot product)



Codice:

```
from numba import jit, prange
import timeit
import numpy as np

def f_n(v1, v2):
    res = 0.0
    for i in range(len(v1)):
        res += v1[i] * v2[i]
    return res

@jit
def g_n(v1, v2):
    res = 0.0
    for i in range(len(v1)):
        res += v1[i] * v2[i]
    return res

@jit(parallel=True)
def h_n(v1, v2):
    res = 0.0
    for i in prange(len(v1)):
        res += v1[i] * v2[i]
    return res

n = 100_000_000
v1 = np.random.rand(n)
v2 = np.random.rand(n)
vout = np.zeros(n)

print(f"Esecuzione di f_n: {timeit.timeit(lambda: f_n(v1, v2), number=1)}")
print(f"Esecuzione di g_n: {timeit.timeit(lambda: g_n(v1, v2), number=1)}")
print(f"Esecuzione di h_n: {timeit.timeit(lambda: h_n(v1, v2), number=1)}")
```

◆ Spiegazione

👉 f_n :

- versione Python normale

👉 g_n :

- versione compilata con **@jit**

👉 h_n :

- usa prange
- viene parallelizzata

✳️ esecuzione su più thread

👉 timeit :

- misura i tempi di esecuzione

◆ Esempio (moltiplicazione matrici)



Codice:

```
from numba import jit, prange
import timeit
import numpy as np

def matmul(M1, M2):
    Mout = np.zeros_like(M1)
    for i in range(M1.shape[0]):
        for j in range(M1.shape[1]):
            s = 0.0
            for k in range(M1.shape[1]):
                Mout[i, j] += M1[i, k] * M2[k, j]
    return Mout

@jit
def matmul_jit(M1, M2):
    Mout = np.zeros_like(M1)
    for i in range(M1.shape[0]):
        for j in range(M1.shape[1]):
            for k in range(M1.shape[1]):
                Mout[i, j] += M1[i, k] * M2[k, j]
    return Mout

@jit(parallel=True)
def matmul_parallel(M1, M2):
    Mout = np.zeros_like(M1)
    for i in prange(M1.shape[0]):
        for j in prange(M1.shape[1]):
            for k in range(M1.shape[1]):
                Mout[i, j] += M1[i, k] * M2[k, j]
    return Mout

n = 250
M1 = np.random.rand(n, n)
M2 = np.random.rand(n, n)

print(f"Esecuzione di matmul: {timeit.timeit(lambda: matmul(M1, M2), number=1)}")
print(f"Esecuzione di matmul_jit: {timeit.timeit(lambda: matmul_jit(M1, M2), number=1)}")
print(f"Esecuzione di matmul_parallel: {timeit.timeit(lambda: matmul_parallel(M1, M2), number=1)}
```

◆ Osservazione

👉 prange :

- permette parallelismo esplicito

👉 parallel=True :

- abilita il parallelismo

✳ insieme permettono di sfruttare più CPU

Ora vediamo un pattern più avanzato: **stencil**

📌 STENCIL

◆ Idea

👉 Uno **stencil** è un pattern in cui:

- ogni elemento di una struttura (es. matrice)
- viene aggiornato usando i valori dei vicini

✳ operazione locale su una griglia

👉 Viene spesso usato per:

- simulazioni
- immagini
- calcolo numerico

◆ Implementazione con Numba

👉 Numba fornisce:

✳ **@stencil**

👉 Permette di:

- definire operazioni sui vicini
- in modo compatto



Codice:

```
import numpy as np
from numba import jit, stencil
from matplotlib import pyplot as plt
from matplotlib import animation

n = 200
arr = np.zeros((n, n))

arr[n//2:n//2+10, n//2:n//2+10] = 1.0
arr[n//3:n//3+20, n//3:n//3+20] = 1.0
arr[n//4:n//4+10, n//4:n//4+10] = 1.0

@stencil(neighborhood=((-1, 1), (-1, 1)))
def stencil_2d(arr):
    s = 0.0
    for i in range(-1, 2):
        for j in range(-1, 2):
            s += arr[i, j]
    return s / 9.0

@jit(parallel=True)
def apply_stencil(arr):
    result = np.zeros_like(arr)
    stencil_2d(arr, out=result)
    return result

fig, ax = plt.subplots()
im = ax.imshow(arr, cmap='grey')
ax.axis('off')

def update(frame):
    global arr
    arr = apply_stencil(arr)
    im.set_data(arr)
    return (im,)

ani = animation.FuncAnimation(fig, update, frames=1000, interval=50, blit=False)
plt.show()
```

◆ Spiegazione

👉 arr :

- matrice iniziale piena di zeri
- alcune sottozone vengono impostate a 1

✳ Sono le “zone attive” da cui parte l'evoluzione

👉 @stencil :

- definisce una funzione che lavora sui vicini
- per ogni elemento guarda un intorno locale

👉 neighborhood= $((-1, 1), (-1, 1))$:

- prende i vicini in un quadrato 3x3
- include l'elemento centrale

👉 stencil_2d :

- scorre i vicini con due cicli
- somma tutti i valori

👉 return s / 9.0 :

- fa la media dei valori nel 3x3

✳ ogni cella diventa la media dei vicini → effetto di diffusione

👉 apply_stencil :

- crea una nuova matrice

- applica lo stencil su tutta la griglia

👉 `stencil_2d(arr, out=result) :`

- applica la funzione a tutta la matrice
- salva il risultato in `result`

👉 `@jit(parallel=True) :`

- esegue l'operazione in parallelo

👉 `update :`

- aggiorna la matrice ad ogni frame
- applica di nuovo lo stencil

👉 `animation :`

- mostra l'evoluzione nel tempo

✳ Si vede la diffusione dei valori nella matrice